Software testing

Software testing is undoubtedly the largest consumer of software quality assurance resources

Testing is certainly not the only type of SQA tool applied to software code. Additional tools are code inspections and code walkthroughs, methods implemented on code printout without actually running the program. These procedures, which are similar to those applied in design inspection and walkthroughs, yield good results in identifying code defects.

“Testing is the process of executing a program with intention of finding errors.”

What is Manual testing?

Manual testing verifies a software product against functional and non-functional requirements manually executed by the QA team. It is performed to discover bugs in software under development.

What does this mean? This means that the QA team will run the product under test on different test environments, precisely as the end users would, to find any deviations from the original software requirements before software deployment. In this process, the team will execute the test cases for checking all the features and possible user scenarios and generate the test reports without the help of any automation software testing tool.

Manual testing usually involves analyzing requirements, creating and executing Test Plan and Test Case, reporting bugs/defects/issues, and checking/ re-verification of fixes.

What is Automation testing?

Automation testing is the process of writing code/test scripts to automate the test execution faster, cheaper, and more efficiently. Appropriate automation tools to develop the test scripts and validate the software are used for comparing the actual results against the expected results.

What does this mean? This means that the QA team will write code or scripts on different tools to run the test cases automatically. This will help the team to determine whether or not the application performs as expected. Even though all processes are performed automatically, automation requires some manual effort to create initial testing scripts.

Automation testing usually involves analysis of requirements, planning, design and development of test scripts and automation tests, execution of the automation test, reporting bugs/defects/issues, and checking/ re-verification of fixes by rerunning the tests.

In manual testing, a human performs the tests step by step, without test scripts. In automated testing, tests are executed automatically via test automation frameworks, along with other tools and software.

The biggest **difference between manual and automation testing** is who executes the test case. In manual testing, the human tester does it. In automation testing, the tool does it.

When to use Manual testing and when to use Automation testing?

	Benefits	Limitations
Manual	freedom to adapt to requirement changes instantly	Human error Expensive for large projects
Automated	Speed Re-usability Immediate results	Complex maintenance if requirement changes frequently Requires time to design tests Check only predefined issues

No one can deny that there are benefits of automation testing over manual testing, especially when saving time and improving test accuracy. But currently, it's simply not possible to have a robust QA process by relying solely on automation. There are QA testing elements that continue to be best handled by human engineers and manual testing

The right mix between automation and manual tests is essential for a successful QA team. It might also be suitable for a tester to switch between manual and automation testing from time to time.

Neither of the two can replace the other entirely. If you care about building good software, you should know that both manual and automation complement each other to perfection, and the team which will embrace both of them will be more efficient.

How to perform Manual Testing

1. Read and understand the software project documentation/guides. Also, study the Application Under Test (AUT) if available.
2. Draft Test cases that cover all the requirements mentioned in the documentation.
3. Review and baseline the test cases with Team Lead, Client (as applicable)
4. Execute the test cases on the AUT
5. Report bugs.
6. Once bugs are fixed, again execute the failing test cases to verify they pass.

Myths of Manual Testing

Following are few common myths and facts related to testing:

Myth: Anyone can do manual testing

Fact: Testing requires many skill sets

Myth: Testing ensures 100% Defect free product

Fact: Testing attempts to find as many defects as possible. Identifying all possible defects is impossible.

Myth: Automated testing is more powerful than manual testing

Fact: 100% test automation cannot be done. Manual Software Testing is also essential.

Myth: Testing is easy

Fact: Testing can be extremely challenging. Testing an application for possible use cases with minimum test cases requires high analytical skills.

Tools to Automate Manual Testing

- Selenium
- QTP
- Jmeter
- Loadrunner
- TestLink
- Quality Center(ALM)

Types of Automated Testing

- Smoke Testing
- Unit Testing
- Integration Testing
- Functional Testing
- Keyword Testing
- Regression Testing
- Data Driven Testing
- Black Box Testing

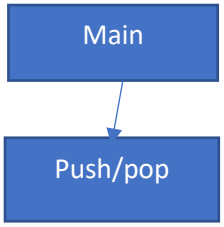
Software testing strategies

Although test methodologies may vary, often greatly, these are applied within the framework of two basic testing strategies:

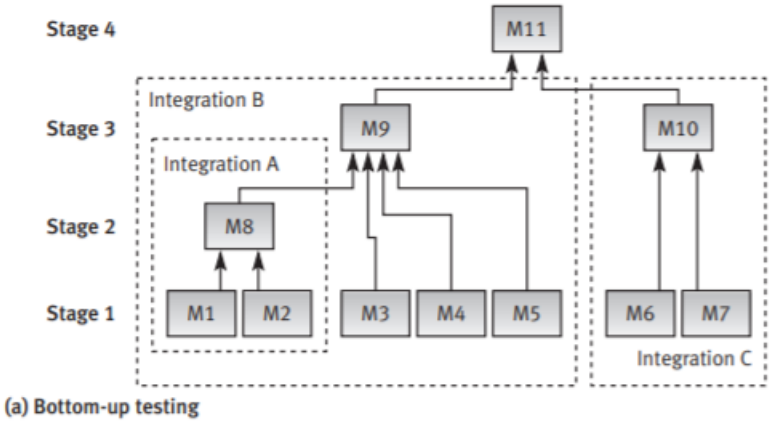
■ To test the software in its entirety, once the completed package is available; otherwise known as “big bang testing”.

■ To test the software piecemeal, in modules, as they are completed (unit tests); then to test groups of tested modules integrated with newly completed modules (integration tests). This process continues until all the package modules have been tested. Once this phase is completed, the entire package is tested as a whole (system test). This testing strategy is usually termed “incremental testing”. Furthermore, incremental testing is also performed according to two basic

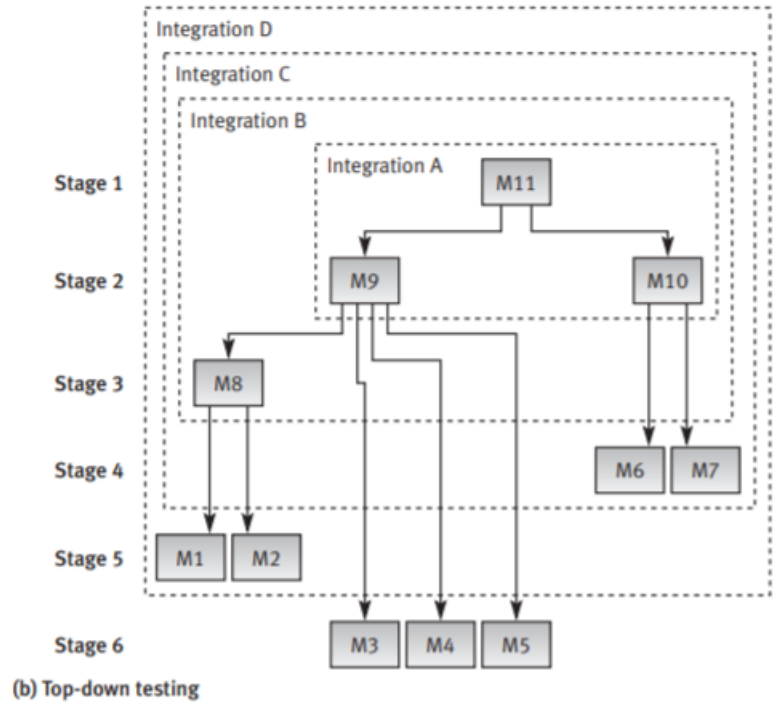
strategies: bottom-up and top-down. Both incremental testing strategies assume that the software package is constructed of a hierarchy of software modules. In top-down testing, the first module tested is the main module, the highest level module in the software structure; the last modules to be tested are the lowest level modules. In bottom-up testing, the order of testing is reversed: the lowest level modules are tested first, with the main module tested last.



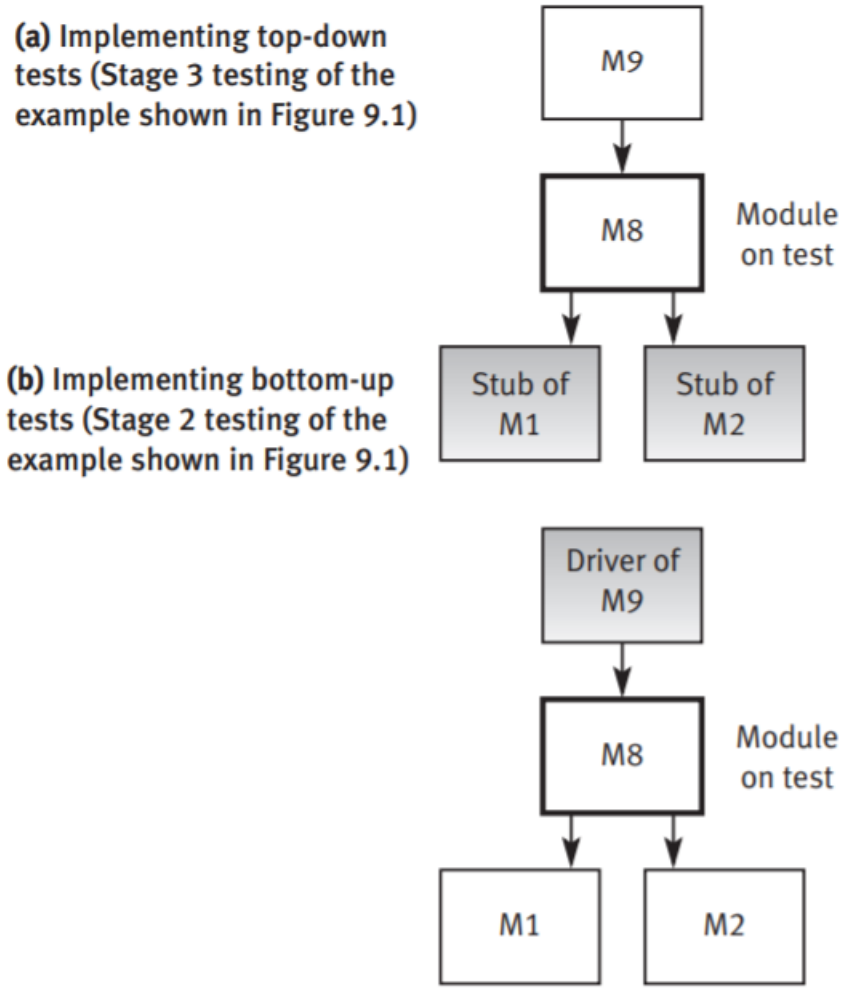
Stubs vs drivers



(a) Bottom-up testing



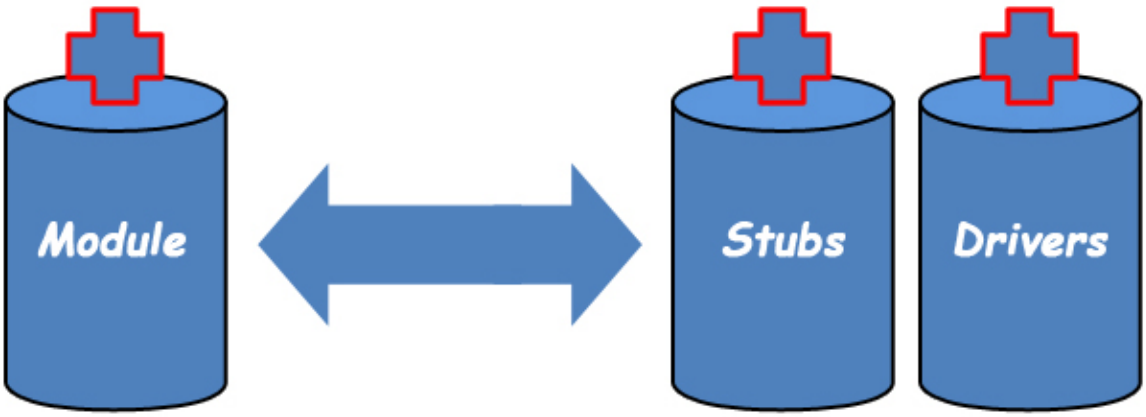
(b) Top-down testing



in incremental testing, a great part of the errors are identified and corrected at an earlier stage of development and testing, which prevents "migration" of escaped defects to a later, more complex stage in the development where their correction would require significantly greater resources. The main disadvantage of incremental testing is the quantity of programming resources required for preparation of stubs and drivers for the unit and integration tests. Another major disadvantage is the need to carry out numerous testing operations for the same program (big bang testing requires only a single testing operation).

It is generally accepted that incremental testing should be preferred despite its disadvantages.

Stubs and drivers are two such elements used in software testing process, which act as a temporary replacement for a module. These are an integral part of software testing process as well as general software development.



n the field of software testing, the term stubs and drivers refers to the replica of the modules, which acts as a substitute to the undeveloped or missing module. The stubs and drives are specifically developed to meet the necessary requirements of the unavailable modules and are immensely useful in getting expected results.

Defining Stubs:
Stubs are used to test modules and are created by the team of testers during the process of Top-Down Integration Testing. With the assistance of these test stubs testers are capable of stimulating the behaviour of the lower level modules that are not yet integrated with the software. Moreover, it helps stimulates the activity of the missing components.

Defining Drivers:
Drivers, like stubs, are used by software testers to fulfil the requirements of missing or incomplete components and modules. These are usually complex than stubs and are developed during Bottom-Up approach of Integration Testing. Drivers can be utilized to test the lower levels of the code, when the upper level of codes or modules are not developed. Drivers act as pseudo codes that are mainly used when the stub modules are ready, but the primary modules are not ready.

Stubs and Drivers: Example

Consider an example of a web application, which consists of 4 modules i.e., Module-A, Module-B, Module-C and Module-D. Each of the following modules is responsible for some specific activity or functionality, as under:

Consider an example of a web application, which consists of 4 modules i.e., Module-A, Module-B, Module-C and Module-D. Each of the following modules is responsible for some specific activity or functionality, as under

Module-A ? Login page of the web application.

Module-B → Home page of the web application.

Module-C → Print Setup.

Module-D → Log out page.

modules A, B, C & D involves the interdependencies of each module over other.

It is always preferable, to perform testing, in parallel, to the development process. Thus, it implies that subsequent testing must be carried out, immediately after the development of the each module.

Module-A will be tested, as soon as, it develops. However, to carry out and validate the testing procedures in respect of module-A, there urges the need of Module-B, which is not yet developed. The expected functionality of the login page (module-A) could be validated, only if it is directed to the home page (Module-B), based on the valid and correct inputs.

But, on the non-availability of theModule-B, it will not be possible to test module-A. These types of circumstances, introduces the stubs & drivers in the process of software testing. A dummy module, representing the basic functionality or feature of the module-B, is being developed, and thereafter, it is being integrated with the module-A, to perform testing, efficiently.

Similarly, stubs and drivers, are used to fulfil the requirements of other modules, such as

Log out page (Module-D), needs to be directed to the login page (Module-A), after successfully logging out from the application. In the event of unavailability of Module-A, stubs and drivers will work as a substitute for it, in order to carry out the testing of module-D.

Stubs	***Drivers***
1. Used in **Top-Down Integration Testing**.	1.Used in **Bottom-Up Integration Testing**.
2. A piece of code that emulates the called function.	2. A piece of code that emulates a calling function.
3. Stubs stimulates the activity of missing and not developed modules.	3. Drivers pass test cases to another code and invoke modules under testing.
4. These are created when high level modules are tested and lower level modules are not developed.	4. These are created when lower level modules are tested and higher level modules are not yet developed.

Software test classifications

■ Black box (functionality) testing. Identifies bugs only according to software malfunctioning as they are revealed in its erroneous outputs. In cases that the outputs are found to be correct, black box testing disregards the internal path of calculations and processing performed.

■ White box (structural) testing. Examines internal calculation paths in order to identify bugs. Although the term “white” is meant to emphasize the contrast between this method and black box testing, the method’s other name – “glass box testing” – better expresses its basic characteristic, that of investigating the correctness of code structure.

Black box testing: (1) Testing that ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions

White box testing: Testing that takes into account the internal mechanism of a system or component

What is System Testing?

System Testing is a level of testing that validates the complete and fully integrated software product. The purpose of a system test is to evaluate the end-to-end system specifications. Usually, the software is only one element of a larger computer-based system. Ultimately, the software is interfaced with other software/hardware systems. System Testing is actually a series of different tests whose sole purpose is to exercise the full computer-based system. System test falls under the **black box testing** category of software testing.
White box testing is the testing of the internal workings or code of a software application. In contrast, black box or System Testing is the opposite. System test involves the external workings of the software from the user’s perspective.

WHITE BOX TESTING

Different paths in a software module are created by the choice in conditional statements, such as IF–THEN–ELSE or DO WHILE or DO UNTIL. Path testing is motivated by the aspiration to achieve complete coverage of a program by testing all its possible paths.

■ “Path coverage” – to plan our test to cover all the possible paths, where coverage is measured by percentage of paths covered.

it is impractical in most cases because of the vast resources required for its performance. Just how costly these applications can be is illustrated in the following example. Let us now calculate the number of possible paths created by a simple module containing 10 conditional statements, each allowing for only two options (e.g., IF–THEN–ALSO and DO WHILE). This simple module contains 1024 different paths. In other words, in order to obtain full path coverage for this module (probably 25–50 lines of code) one should prepare at least 1024 test cases, one for each possible path. A straightforward calculation of the number of test cases required to test a software package that contains 100 modules of similar complexity (a total of 102 400 test cases) readily indicates the impracticality of wide use of path testing. Hence, its application is directed mainly to high risk software modules, where the costs of failure resulting from software error fully warrant the costs of path testing. This situation has encouraged development of an alternative yet weaker coverage concept – line coverage

The code for the program would look something like this:

```
input x
if x > 5 then
return A
else return B
```

In order for path coverage testing to effectively "cover all paths," the two test cases must be run, with x greater than 5 and x less than or equal to 5.

■ "Line coverage" – to plan our tests to cover all the program code lines, where coverage is measured by percentage of lines covered. The line coverage concept requires that, for full line coverage, every line of code be executed at least once during the process of testing.

```
if(cond) {
   line1();
   line2();
   line3();
   line4();
} else {
   line5();
}
```

If your test only exercises the cond being true and never runs the else branch you have:

- 4 out of 5 lines covered
- 1 out of 2 branches covered

- Consider code sample 4.1 :

```
READ X
READ Y
I F X>Y THEN Z = 0
ENDIF
```

Code sample 4.1

- To achieve 100% statement coverage of this code segment just one test case is required, one which ensures that variable A contains a value that is greater than the value of variable Y, for example, X = 12 and Y = 10.

In the source code, when each occurring condition is evaluated for both true and false states, then the **Condition Coverage** for the code is said to be complete.

```
Add (int a, int b)
{
        If (b > a)
        {
          b = b - a
          Print b
        }
        If (a > b)
        {
            b = a – b
            Print b
        }
                Else Print '0'
}
```

- **For example,** in the above code if value sets (2, 3) and (4, 2) are used then Condition Coverage would be 100%. When data set (2, 3) is used then (b > a) evaluates to true and (a > b) evaluates to false. Similarly, when data set (4, 2) is used then (b > a) evaluates to false and (a > b) evaluates to true.
- Thus both the conditions have both the values i.e true and false covered. Hence the Condition Coverage would be 100%.

This methodology aims at ensuring that every branch appearing in each conditional structure gets executed in source code. For instance, in the above code, all the 'If' statements and any

accompanying 'Else' statement should all be covered by the test for a 100% **Branch Coverage**.

Branch Coverage Vs Condition Coverage
Branch Coverage is often confused with Condition Coverage, however, the two are different.

Let's understand this with a simple example.
If (a >0) & (b >0)

Then Print "Hello"

Else Print "Bye"

Let us write down the data set needed for complete Branch Coverage:
(1, 1) – In this case, 'a' and 'b' both are true, so the If condition gets executed.
(1, 0) – In this case, 'a' is true and 'b' would be false, so the Else part of the code is executed.

As we know the purpose of Branch Coverage is to get every branch executed at least once and this purpose is achieved.

Condition Coverage:
(1, 0) – In this case, 'a' is true and 'b' would be false.
(0, 1) – In this case, 'a' is false and 'b' would be true.

The purpose of Condition Coverage is to get each of true and false for every condition executed and this purpose is achieved here.

TOOLS for code coverage

- Parasoft JTest (java)
- Testwell CTC++
- Cobertura (java)
- JaCoCo (java)

Coverage.py is one of the most popular code coverage tools for Python. It uses code analysis tools and tracing hooks provided in Python standard library to measure coverage

Pytest-cov is a Python plugin to generate coverage reports

```
Add (int a, int b)
{
        If (b > a)
        {
          b = b - a
          Print b
        }
        If (a > b)
        {
            b = a – b
            Print b
```

```
        }
                Else Print '0'
}
```

- **For example,** in the above code if value sets (2, 3), (4, 2), (1, 1) are used then Branch Coverage would be 100%. When data set (2, 3) is used then (b > a) and the first 'If' branch gets executed. Similarly, when data set (4, 2) is used then (a > b) evaluates to true and the second 'If' branch gets executed.
- Then with the data set (1, 1) the 'Else' branch evaluates to true and gets executed. Thereby, ensuring 100% Branch Coverage.

Advantages and disadvantages of white box testing

The main advantages of white box testing are:

■ Direct statement-by-statement checking of code enables determination of software correctness as expressed in the processing paths, including whether the algorithms were correctly defined and coded.

■ It ascertains quality of coding work and its adherence to coding standards.

The main disadvantages of white box testing are:

■ The vast resources utilized, much above those required for black box testing of the same software package.

■ The inability to test software performance in terms of availability (response time), reliability, load durability, and other testing classes related to operation, revision and transition factors. The characteristics of white box testing limit its use to software modules of very high risk and very high cost of failure, where it is highly important to identify and fully correct as many of the software errors as possible.

Black box testing

- Equivalence classes for output correctness tests
- Equivalence class partitioning is a black box method aimed at increasing the efficiency of testing and, at the same time, improving coverage of potential error conditions. An equivalence class (EC) is a set of input variable values that produce the same output results or that are processed identically.
- EC boundaries are defined by a single numeric or alphabetic value, a group of numeric or alphabetic values, a range of values, and so on. An EC that contains only valid states is defined as a "valid EC", whereas an EC that contains only invalid states is defined as an "invalid EC". In cases where a program's input is provided by several variables, valid and invalid ECs should be defined for each variable.
- According to the equivalence class partitioning method, test cases are defined so that each valid EC and each invalid EC are included in at least one test case. Test cases are defined

separately for the valid and invalid ECs. In defining a test case for the valid ECs, we try to cover as many as possible "new" ECs

Compared to the use of a random sample of test cases, equivalence classes save testing resources because they eliminate duplication of the test cases defined for each EC. Importantly, as the equivalence class method is a black box method, equivalence class partitioning is based on software specification documentation, not on the code. Systematic constructing of equivalence classes for a program's input variables increases the coverage of possible valid and error conditions of input and thus improves the testing plan's effectiveness.

Example – the Golden Splash Swimming Center The following example illustrates the definition of (valid and invalid) equivalence classes and the corresponding test case values. The software module in question calculates entrance ticket prices for the Golden Splash Swimming Center.

The Center's ticket price depends on four variables: day (weekday, weekend), visitor's status (OT = one time, M = member), entry hour (6.00–19.00, 19.01–24.00) and visitor's age (up to 16, 16.01–60, 60.01–120). The entrance ticket price table is shown in Table 9.7.

Table 9.7: Entrance ticket price table – the Golden Splash Swimming Center

	Mon, Tue, Wed, Thurs, Fri				**Sat, Sun**			
Visitor's status	OT	OT	M	M	OT	OT	M	M
Entry hour	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00	6.00–19.00	19.01–24.00
	Ticket prices – $							
Visitor's age								
0.0–16.00	**5.00**	**6.00**	**2.50**	**3.00**	**7.50**	**9.00**	**3.50**	**4.00**
16.01–60.00	**10.00**	**12.00**	**5.00**	**6.00**	**15.00**	**18.00**	**7.00**	**8.00**
60.01–120.00	**8.00**	**8.00**	**4.00**	**4.00**	**12.00**	**12.00**	**5.50**	**5.50**

A total of 15 test cases cover all the defined ECs

Table 9.8: Equivalence classes – the Golden Splash Swimming Center ticket price module

Variable	Valid equivalence classes	Representing values		Invalid equivalence classes	Representing values for invalid ECs
		Values for valid ECs	Boundary values		
Day of week	(1) Mon, Tue, Wed, Thurs, Fri	Mon		(1) Any alpha-numeric value (not a day)	Mox
	(2) Sat, Sun	Sat			
Visitor's status	(1) OT (2) M	OT M		Other than OT or M	88
Entry hour	(1) 6.00–19.00 (2) 19.01–24.00	7.55 20.44	6.00, 19.00 19.01, 24.00	(1) Hours < 6.00 (2) Any alpha-numeric values (not time)	4.40 &@
Visitor's age	(1) 0.0–16.00 (2) 16.01–60.00 (3) 60.01–120.00	8.4 42.7 65.0	0.0, 16.00 16.01, 60.00 60.01, 120.00	(1) Any alpha-numeric value (not an age) (2) Ages > 120.0	TTR 150.1

Table 9.9: Test cases – the Golden Splash Swimming Center ticket price module

Test case type	Test case no.	Day of week	Visitor's status	Entry hour	Visitor's age	Test case results
For valid ECs	1	Mon	OT	7.55	8.4	$5.00
	2	Sat	M	20.44	42.7	$8.00
	3	Sat	M	22.44	65.0	$5.50
	4	Sat	M	6.00	0.0	$3.50
	5	Sat	M	19.00	16.00	$3.50
	6	Sat	M	19.01	16.01	$8.00
	7	Sat	M	19.01	60.00	$8.00
	8	Sat	M	24.00	60.01	$5.50
	9	Sat	M	24.00	120.0	$5.50
For invalid ECs	10	Mox	OT	7.55	8.4	Invalid day
	11	Mon	88	7.55	8.4	Invalid visitor status
	12	Mon	OT	4.40	8.4	Invalid entry hour
	13	Mon	OT	&@	8.4	Invalid entry hour
	14	Mon	OT	7.55	TTR	Invalid visitor age
	15	Mon	OT	7.55	150.1	Invalid visitor age

Advantages and disadvantages of black box testing

The main advantages of black box testing are:

■ Black box testing allows us to carry out the majority of testing classes, most of which can be implemented solely by black box tests. Of the test classes unique to black box testing, of special importance are system performance tests such as load tests and availability tests.

■ For testing classes that can be carried out by both white and black box tests, black box testing requires fewer resources than those required for white box testing of the same software package. The main disadvantages of black box testing are:

■ Possibility that coincidental aggregation of several errors will produce the correct response for a test case, and prevent error detection. In other words, black box tests do not readily identify cases of errors that counteract each other to accidentally produce the correct output.

■ Absence of control of line coverage. In cases where black box testers wish to improve line coverage, there is no easy way to specify the parameters of the test cases required to improve coverage. Consequently, black box tests may not execute a substantial proportion of the code lines, which are not covered by a set of test cases.

■ Impossibility of testing the quality of coding and its strict adherence to the coding standards.

Equivalence class testing and boundary value testing (detailed)

What is Boundary Testing?

Boundary testing is the process of testing between extreme ends or boundaries between partitions of the input values.

1. Minimum
2. Just above the minimum
3. A nominal value
4. Just below the maximum
5. Maximum

- Boundary Testing comes after the Equivalence Class Partitioning.

Equivalence Partitioning

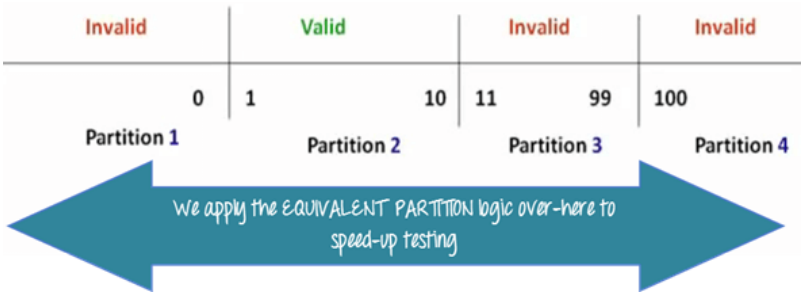
Equivalence Partitioning or Equivalence Class Partitioning is type of black box testing technique which can be applied to all levels of software testing like unit, integration, system, etc. In this technique, input data units are divided into equivalent partitions that can be used to derive test cases which reduces time required for testing because of small number of test cases.

- Let's consider the behavior of Order Pizza Text Box Below
- Pizza values 1 to 10 is considered valid. A success message is shown.
- While value 11 to 99 are considered invalid for order and an error message will appear, **"Only 10 Pizza can be ordered"**

Here is the test condition

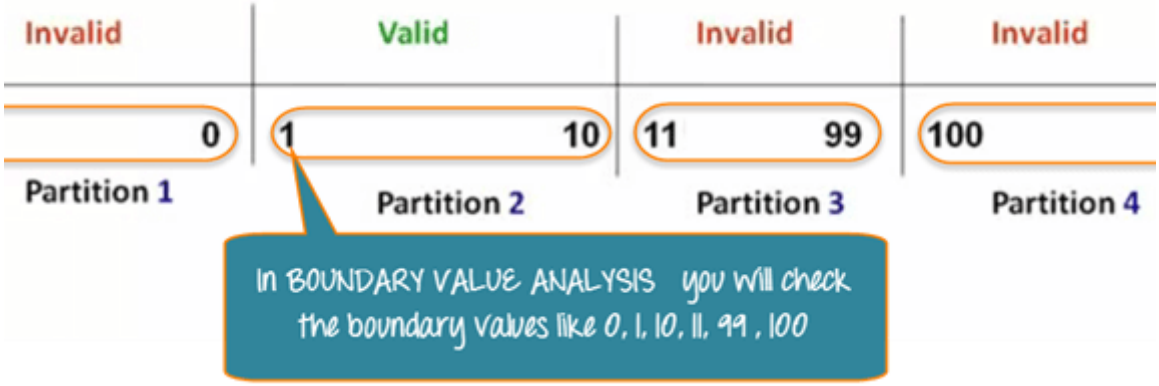
1. Any Number greater than 10 entered in the Order Pizza field(let say 11) is considered invalid.
2. Any Number less than 1 that is 0 or below, then it is considered invalid.
3. Numbers 1 to 10 are considered valid
4. Any 3 Digit Number say -100 is invalid.

We cannot test all the possible values because if done, the number of test cases will be more than 100. To address this problem, we use equivalence partitioning hypothesis where we divide the possible values of tickets into groups or sets as shown below where the system behavior can be considered the same.



The divided sets are called Equivalence Partitions or Equivalence Classes. Then we pick only one value from each partition for testing. The hypothesis behind this technique is **that if one condition/value in a partition passes all others will also pass**. Likewise**, if one condition in a partition fails, all other conditions in that partition will fail**.

Boundary Value Analysis- in Boundary Value Analysis, you test boundaries between equivalence partitions



In our earlier equivalence partitioning example, instead of checking one value for each partition, you will check the values at the partitions like 0, 1, 10, 11 and so on. As you may observe, you test values at **both valid and invalid boundaries**. Boundary Value Analysis is also called **range checking**.

Equivalence partitioning and boundary value analysis(BVA) are closely related and can be used together at all levels of testing.

What is Traceability Matrix? (TM)

A Traceability Matrix is a document that co-relates any two-baseline documents to track the requirements and to check the current project requirements are met.

Requirement Traceability Matrix (RTM) is a document that maps and traces user requirement with test cases. It captures all requirements proposed by the client and requirement traceability in a single document, delivered at the conclusion of the Software developement life cycle. The main purpose of Requirement Traceability Matrix is to validate that all requirements are checked via test cases such that no functionality is unchecked during Software testing

The main agenda of every tester should be to understand the client's requirement and make sure that the output product should be defect-free. To achieve this goal, every QA should understand the requirement thoroughly and create positive and negative test cases.

A simple way is to trace the requirement with its corresponding test scenarios and test cases. This merely is termed as 'Requirement Traceability Matrix.'

The traceability matrix is typically a worksheet that contains the requirements with its all possible test scenarios and cases and their current state, i.e. if they have been passed or failed. This would help the testing team to understand the level of testing activities done for the specific product.

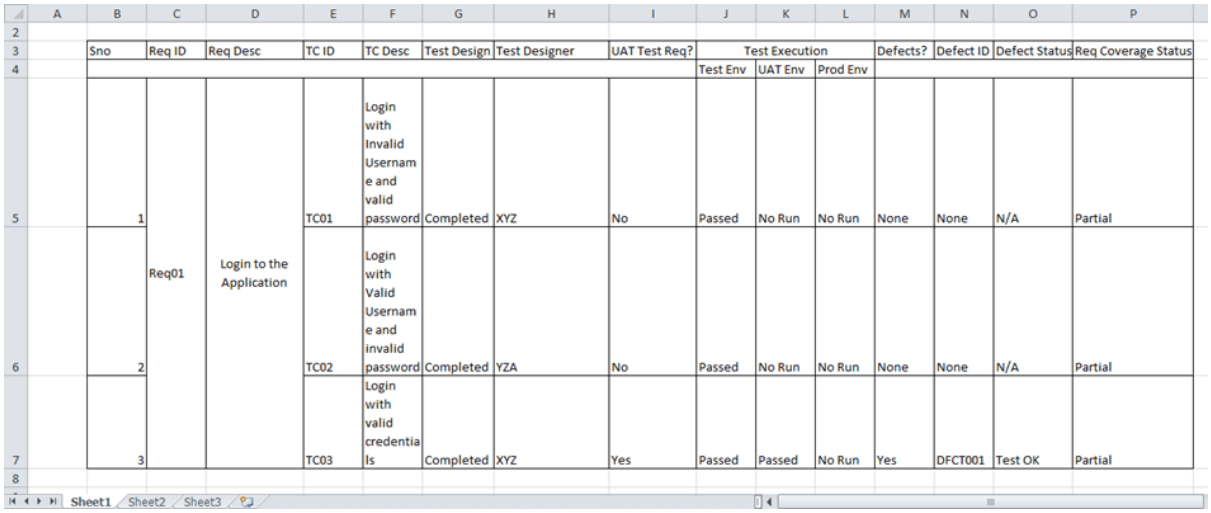
Which Parameters to include in Requirement Traceability Matrix?

- Requirement ID
- Requirement Type and Description
- Test Cases with Status

Req No	Req Desc	Testcase ID	Status
123	Login to the application	TC01,TC02,TC03	TC01-Pass TC02-Pass
345	Ticket Creation	TC04,TC05,TC06, TC07,TC08,TC09 TC010	TC04-Pass TC05-Pass TC06-Pass TC06-Fail TC07-No Run
456	Search Ticket	TC011,TC012, TC013,TC014	TC011-Pass TC012-Fail TC013-Pass TC014-No Run

Above is a sample requirement traceability matrix.

But in a typical software testing project, the traceability matrix would have more than these parameters.



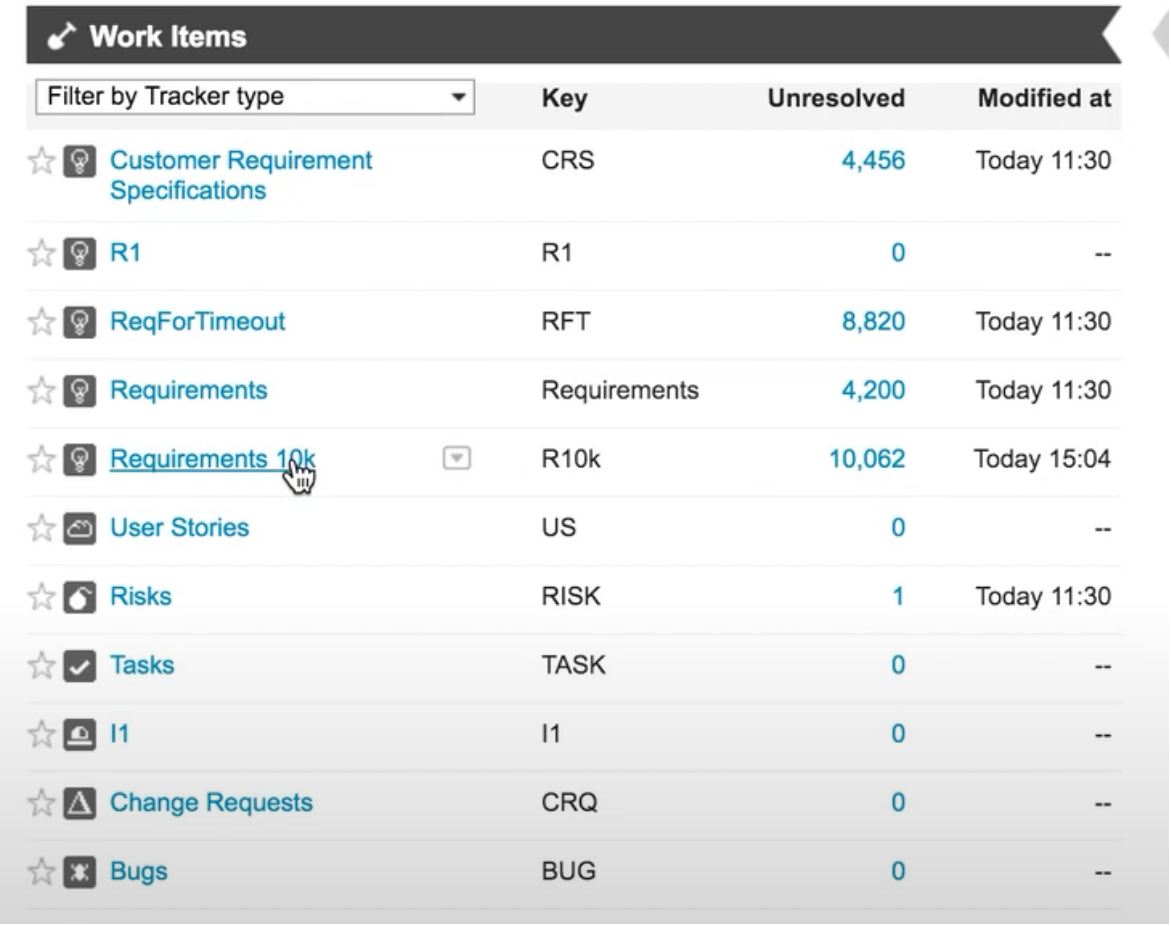
Sno	Req ID	Req Desc	TC ID	TC Desc	Test Design	Test Designer	UAT Test Req?	Test Execution			Defects?	Defect ID	Defect Status	Req Coverage Status
								Test Env	UAT Env	Prod Env				
1	Req01	Login to the Application	TC01	Login with Invalid Username and valid password	Completed	XYZ	No	Passed	No Run	No Run	None	None	N/A	Partial
2			TC02	Login with Valid Username and invalid password	Completed	YZA	No	Passed	No Run	No Run	None	None	N/A	Partial
3			TC03	Login with valid credentials	Completed	XYZ	Yes	Passed	Passed	No Run	Yes	DFCT001	Test OK	Partial

As illustrated above, a requirement traceability matrix can:

- Show the requirement coverage in the number of test cases
- Design status as well as execution status for the specific test case
- If there is any User Acceptance test to be done by the users, then UAT status can also be captured in the same matrix.
- The related defects and the current state can also be mentioned in the same matrix.

Apart from maintaining an excel separately. A testing team can also opt for requirements tracing available Test Management Tools. Code beamer is an award winning tool for managing RTM.

CODE BEAMER



CodeBeamer operates with User Accounts. A User must have an account to log in.

Managing Large Numbers of Projects

When working with many projects, it is useful to sort related projects into *Working Sets*. Projects could be grouped according to line of business, or functions such as *Research and Development* or *Finance*.

Roles

Roles reflect hierarchical or task-based team structures in a project. A role's access permission's scope is project-wide. For example a role can be *Developer* or *Designer* where users in the Developer role are granted *write* access on the SCM system but users in the Designer role are not.

Custom Roles

Custom roles are useful when the standard codeBeamer roles (Customer, Developer - Extern, Developer - Intern, Stakeholder) are insufficient to organize your team structure, and you need a new role e.g. Architect, with specific access privileges. Roles' permissions and custom roles are specific to the project in which they are created. Custom roles can be created and default, standard roles can be edited in any project, assuming the role-editor has sufficient permissions on the project.

If you create a new role in an existing project, without using a template for the new role, the new role will not have permissions for any project artifacts created before the role.

Role and Group Based Access Control

Role and Group Based Access Control is useful to manage access control for medium or large numbers of users.

Codeless automation

- Software Testing has traversed a long path from manual testing to record-and-playback sophisticated testing tools, then to test automation, and now to codeless automated testing.
- Codeless automated testing means executing automation testing without using any script. Writing automated scripts could be challenging for QA engineers as the code required to be written in every suitable programming language is arduous and time-consuming.
- With codeless automated testing, you can minimize the testing time over repetitive test cases without any coding efforts. There are many codeless testing tools available in the market, which encompass a set of built-in features without any complicated coding. Some of the significant features include:
- Drag and Drop interface.
- Record and Playback.
- Integrations of ML or AI technologies.
- Lower maintenance.

Why Should You Consider Codeless Automated Testing?

- Performing codeless automated testing is not completely code-free; however, there are many reasons why you must consider implementing codeless test automation into the software testing process.
- Codeless testing facilitates easy testing with the team spending little effort on automation.
- Codeless automation test tools increase test coverage.
- With codeless automation, even the complicated test scripts are simple to change and manage.
- Codeless testing is a quick way to automate the test scenario.
- Codeless testing has a user-friendly interface.
- Codeless testing is cost-effective.

Best Codeless Testing Tools For 2021

Katalon studio

- Katalon studio is ranked among the top codeless testing tools of 2020. It is a popular choice for its extensive range of test abilities across desktop, mobile, web, apis, and applications. Its unique features are simple-to-use and best for experienced and beginners QA engineers. The credits sum up why it is at the top of our list.

Leapwork

- Leapwork is an advanced cloud-based, codeless selenium automation testing platform. It supports the testing of many forms of apps, including SAP, windows, and web. With LEAPWORK, you can enable your agile project team to speed up the development and implementation of the QA automated testing procedure.
- Designed for both desktop apps and web apps, this tool is built with the entire software development team in mind.

testCraft

- This is one of the most incredible codeless testing tools, which was introduced as an E2E (end-to-end) test automation platform for continuous and regression tests. Similar to other codeless testing tools with the special feature of Record and Playback, this tool also offers a test script with inputs coded beforehand that must be modified afterward. Tweaking some codes might break the testing script.

Selenium

- This codeless selenium automation tool allows you to automate web applications with everything that Selenium offers without needing to write codes. Selenium IDE uses the record & playback feature to support codeless automated testing.

Unit testing

Static unit testing

- In static unit testing, a programmer does not execute the unit; instead, the code is examined over all possible behaviors that might arise during run time.
- Static unit testing is also known as non-execution-based unit testing, whereas dynamic unit testing is execution based. In static unit testing, the code of each unit is validated against requirements of the unit by reviewing the code.
- It may be noted that static unit testing is not an alternative to dynamic unit testing. A programmer performs both kinds of tests. In practice, partial dynamic unit testing is performed concurrently with static unit testing. If the entire dynamic unit testing has been performed and a static unit testing identifies significant problems, the dynamic unit testing must be repeated. As a result of this repetition, the development schedule may be affected. To minimize the probability of such an event, it is required that static unit testing be performed prior to the final dynamic unit testing.

In static unit testing, code is reviewed by applying techniques commonly known as inspection and walkthrough.

- Inspection: It is a step-by-step peer group review of a work product, with each step checked against predetermined criteria.
- Walkthrough: It is a review where the author leads the team through a manual or simulated execution of the product using predefined scenarios.

Dynamic unit testing

- Stubs drivers

Selection of test data

- Control flow testing
- Data flow testing
- **Control flow testing** is a form of white-box testing where the implementation of the code is known to the tester. The development team often performs control flow testing. This process determines and observes the execution paths of a program in a structured way. We use this technique to develop the test cases.

We determine the different flows of the program before developing test cases for each case, which are then tested. This testing technique is mostly used in unit testing.

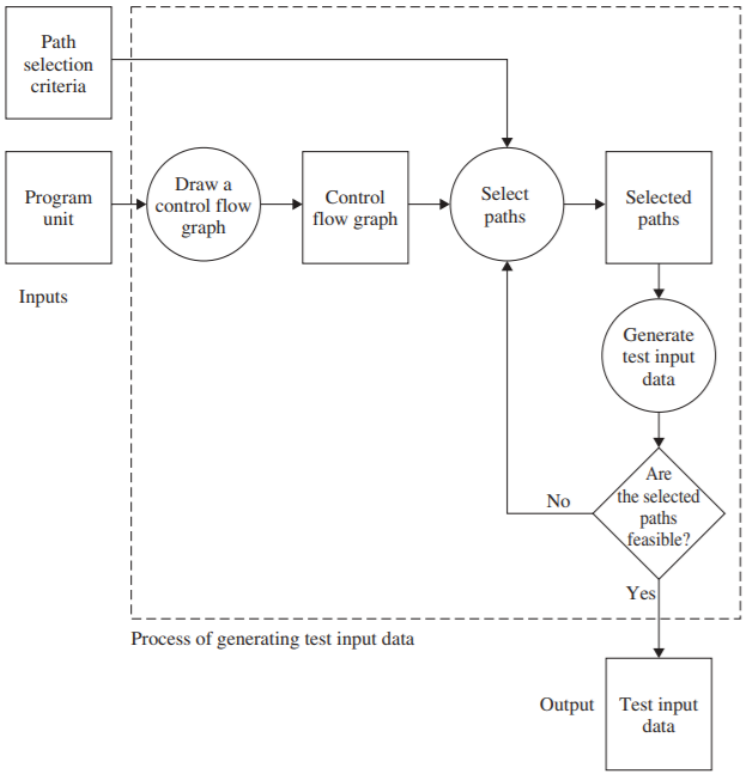
- Two kinds of basic statements in a program unit are assignment statements and conditional statements. An assignment statement is explicitly represented by using an assignment symbol, " = ", such as x = 2*y;, where x and y are variables. Program conditions are at the core of conditional statements, such as if(), for() loop, while() loop, and goto. As an example, in if(x! = y), we are testing for the inequality of x and y. In the absence of conditional statements, program instructions are executed in the sequence they appear. The idea of successive execution of instructions gives rise to the concept of control flow in a program unit. Conditional statements alter the default, sequential control flow in a program unit. In fact, even a small number of conditional statements can lead to a complex control flow structure in a program.
- Control flow testing is a kind of structural testing, which is performed by programmers to test code written by them
- For a given set of input data, the program unit executes a certain path. For another set of input data, the unit may execute a different path. The main idea in control flow testing is to appropriately select a few paths in a program unit and observe whether or not the selected paths produce the expected outcome. By executing a few paths in a program unit, the programmer tries to assess the behavior of the entire program unit

Activities:

Inputs: The source code of a program unit and a set of path selection criteria are the inputs to a process for generating test data. In the following, two examples of path selection criteria are given.

Example. Select paths such that every statement is executed at least once.

Example. Select paths such that every conditional statement, for example, an if() statement, evaluates to true and false at least once on different occasions. A conditional statement may evaluate to true in one path and false in a second path.



- **Generation of a Control Flow Graph:** A control flow graph (CFG) is a detailed graphical representation of a program unit. The idea behind drawing a CFG is to be able to visualize all the paths in a program unit.
- **Selection of Paths:** Paths are selected from the CFG to satisfy the path selection criteria, and it is done by considering the structure of the CFG.
- **Generation of Test Input Data:** A path can be executed if and only if a certain instance of the inputs to the program unit causes all the conditional statements along the path to evaluate to true or false as dictated by the control flow. Such a path is called a feasible path. Otherwise, the path is said to be infeasible. It is essential to identify certain values of the inputs from a given path for the path to execute.
- **Feasibility Test of a Path:** The idea behind checking the feasibility of a selected path is to meet the path selection criteria. If some chosen paths are found to be infeasible, then new paths are selected to meet the criteria. An **infeasible path** in software testing is a path that is impossible to achieve with any set of inputs.

What paths do I select for testing?

- The concept of path selection criteria is useful is answering the above question
- well-known path selection criteria:
- • Select all paths.
- • Select paths to achieve complete statement coverage.
- • Select paths to achieve complete branch coverage.
- • Select paths to achieve predicate coverage.

Data Flow Testing is a specific strategy of software testing that focuses on data variables and their values. It makes use of the control flow graph. When it comes to categorization Data flow testing will can be considered as a type of white box testing and structural types of testing.

Advantages of Data Flow Testing:

Data Flow Testing is used to find the following issues-

- To find a variable that is used but never defined,
- To find a variable that is defined but never used,
- To find a variable that is defined multiple times before it is use,
- Deallocating a variable before it is used.

Disadvantages of Data Flow Testing

- Time consuming and costly process
- Requires knowledge of programming languages

1. read x;	
2. If(x>0)	(1, (2, t), x), (1, (2, f), x)
3. a= x+1	(1, 3, x)
4. if (x<=0) {	(1, (4, t), x), (1, (4, f), x)
5. if (x<1)	(1, (5, t), x), (1, (5, f), x)
6. x=x+1; (go to 5)	(1, 6, x)
else	
7. a=x+1	(1, 7, x)
8. print a;	(6,(5, f)x), (6,(5,t)x)
	(6, 6, x)
	(3, 8, a), (7, 8, a).

In this code, a have a total 8 statements, and we will choose a path which covers all the 8 statements. As it is evident in the code, we cannot cover all the statements in a single path because if statement 2 is true then statements 4, 5, 6, 7 not covered, and if statement 4 is true then statement 2 and 3 are not covered.

So, we are taking two paths to cover all the statements.

For the second path, we take the value of x is 1

2. Set x= -1

Path = 1, 2, 4, 5, 6, 5, 6, 5, 7, 8

Output = 2

1. x= 1

Path - 1, 2, 3, 8

Output = 2

When we set value of x as 1 first it come on step 1 to read and assign the value of x (we took 1 in path) then come on statement 2 (x>0 (we took 2 in path)) which is true and it comes on statement 3 (a= x+1 (we took 3 in path)) at last it comes on statement 8 to print the value of x (output is 2).

\

Associations

- **(1, (2, t), x), (1, (2, f), x)-** This association is made with statement 1 (read x;) and statement 2 (If(x>0)) where x is defined at line number 1, and it is used at line number 2 so, x is the variable.
  Statement 2 is logical, and it can be true or false that's why the association is defined in two ways; one is (1, (2, t), x) for true and another is (1, (2, f), x) for false.

- **(1, 3, x)-** This association is made with statement 1 (read x;) and statement 3 (a= x+1) where x is defined in statement 1 and used in statement 3. It is a computation use.

- **(1, (4, t), x), (1, (4, f), x)-** This association is made with statement 1 (read x;) and statement 4 (If(x<=0)) where x is defined at line number 1 and it is used at line number 4 so x is the variable. Statement 4 is logical, and it can be true or false that's why the association is defined in two ways one is (1, (4, t), x) for true and another is (1, (4, f), x) for false.

- **(1, (5, t), x), (1, (5, f), x)-** This association is made with statement 1 (read x;) and statement 5 (if (x<1)) where x is defined at line number 1, and it is used at line number 5, so x is the variable.
  Statement 5 is logical, and it can be true or false that's why the association is defined in two ways; one is (1, (5, t), x) for true and another is (1, (5, f), x) for false.

- **(1, 6, x)-** This association is made with statement 1 (read x;) and statement 6 (x=x+1). x is defined in statement 1 and used in statement 6. It is a computation use.

- **(1, 7, x)-** This association is made with statement 1 (read x) and statement 7 (a=x+1). x is defined in statement 1 and used in statement 7 when statement 5 is false. It is a computation use.

- **(6, (5, f) x), (6, (5, t) x)-** This association is made with statement 6 (x=x+1;) and statement 5 if (x<1) because x is defined in statement 6 and used in statement 5. Statement 5 is logical, and it can be true or false that's why the association is defined in two ways one is (6, (5, f) x) for true and another is (6, (5, t) x) for false. It is a predicted use

- **(6, 6, x)-** This association is made with statement 6 which is using the value of variable x and then defining the new value of x.
  x=x+1
  x= (-1+1)
  Statement 6 is using the value of variable x that is ?1 and then defining new value of x [x= (-1+1) = 0] that is 0.

- **(3, 8, a)-** This association is made with statement 3(a= x+1) and statement 8 where variable a is defined in statement 3 and used in statement 8.

- **(7, 8, a)-** This association is made with statement 7(a=x+1) and statement 8 where variable a is defined in statement 7 and used in statement 8.

Predicate use (p-use)

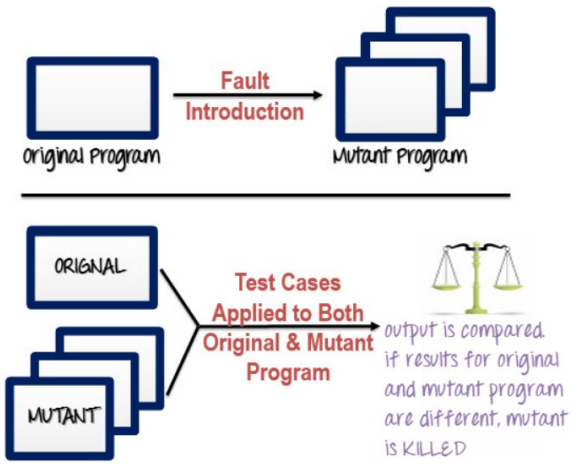
- If the value of a variable is used to decide an execution path is considered as predicate use (p-use). In control flow statements there are two

- Statement 4 if (x<=0) is predicate use because it can be predicate as true or false. If it is true then if (x<1),6x=x+1; execution path will be executed otherwise, else path will be executed.
- Computation use (c-use)
- If the value of a variable is used to compute a value for output or for defining another variable.
- **Statement 3** a= x+1 (1, 3, x)
  Statement 7 a=x+1 (1, 7, x)
  Statement 8 print a (3, 8, a), (7, 8, a).

After collecting these groups, (By examining each point whether the variable is used at least once or not) tester can see all statements and variables are used. The statements and variables which are not used but exist in the code, get eliminated from the code.

MUTATION TESTING (evaluate the quality of already existing software tests)

- **Mutation Testing** is a type of software testing in which certain statements of the source code are changed/mutated to check if the test cases are able to find errors in source code. The goal of Mutation Testing is ensuring the quality of test cases in terms of robustness that it should fail the mutated source code.
- A mutation of a program is a modification of the program created by introducing a single, small, legal syntactic change in the code. A modified program so obtained is called a mutant.
- The term mutant has been borrowed from biology. Some of these mutants are equivalent to the original program, whereas others are faulty. A mutant is said to be killed when the execution of a test case causes it to fail and the mutant is considered to be dead.
- Some mutants are equivalent to the given program, that is, such mutants always produce the same output as the original program
- Mutation was originally proposed in 1971 but lost fervor due to the high costs involved. Now, again it has picked steam and is widely used for languages such as Java and XML.



Step 1: Faults are introduced into the source code of the program by creating many versions called mutants. Each mutant should contain a single fault, and the goal is to cause the mutant version to fail which demonstrates the effectiveness of the test cases.

Step 2: Test cases are applied to the original program and also to the mutant program. A Test Case should be adequate, and it is tweaked to detect faults in a program.

Step 3: Compare the results of an original and mutant program.

Step 4: If the original program and mutant programs generate the different output, then that the mutant is killed by the test case. Hence the test case is good enough to detect the change between the original and the mutant program.

Step 5: If the original program and mutant program generate the same output, Mutant is kept alive. In such cases, more effective test cases need to be created that kill all mutants.

How to change?

Operand replacement operators	Expression Modification Operators	Statement modification Operators
Replace the operand with another operand (x with y or y with x) or with the constant value.	Replace an operator or insertion of new operators in a program statement.	Programmatic statements are modified to create mutant programs.

Operand replacement operators	Expression Modification Operators	Statement modification Operators
Example- If(x>y) replace x and y values If(5>y) replace x by constant 5	Example- If(x==y) We can replace == into >= and have mutant program as If(x>=y) and inserting ++ in the statement If(x==++y)	Example- Delete the else part in an if-else statement Delete the entire if-else statement to check how a program behaves Some of sample mutation operators: • GOTO label replacement • Return statement replacement • Statement deletion • Unary operator insertion (Like – and ++) • Logical connector replacement • Comparable array name replacement • Removing of else part in the if-else statement • Adding or replacement of operators • Statement replacement by changing the data

- Mutation testing is extremely time consuming and complicated to execute manually. To speed up the process, it is advisable to go for automation tools. Automation tools reduce the cost of testing as well.
- List of tools available –
- Stryker — Javascript
- PIT — Java or Kotlin

The mutation score is defined as the percentage of killed mutants with the total number of mutants.

- Mutation Score = (Killed Mutants / Total number of Mutants) * 100

Traditional test coverage (i.e line, statement, branch, etc.) measures only which code is **executed** by your tests. It does **not** check that your tests are actually able to **detect faults** in the executed code. It is therefore only able to identify code that is definitely **not tested**.

The objective of mutation testing is:

- To identify pieces of code that are not tested properly.
- To identify hidden defects that can't be detected using other testing methods.
- To discover new kinds of errors or bugs.
- Types of Mutation Testing:

Mutation testing is basically of 3 types:

Value Mutations:

In this type of testing the values are changed to detect errors in the program. Basically a small value is changed to a larger value or a larger value is changed to a smaller value. In this testing basically constants are changed.

Example:

Initial Code:

- int mod = 1000000007;
- int a = 12345678;
- int b = 98765432;
- int c = (a + b) % mod;
- Changed Code:
- int mod = 1007;
- int a = 12345678;
- int b = 98765432;
- int c = (a + b) % mod;

Decision Mutations:

In decisions mutations are logical or arithmetic operators are changed to detect errors in the program.

- Example:
- Initial Code:
- if(a < b)
- c = 10;
- else
- c = 20;

Statement Mutations:

In statement mutations a statement is deleted or it is replaces by some other statement.

Example:

Initial Code:

if(a < b)

c = 10;

else

c = 20;

Changed Code:

if(a < b)

d = 10;

else

d = 20;

A mutation score for a set of test cases is the percentage of nonequivalent mutants killed by the test suite. The test suite is said to be mutation adequate if its mutation score is 100%.

Advantages of Mutation Testing:

- It brings a good level of error detection in the program.
- It discovers ambiguities in the source code.

Disadvantages of Mutation Testing:

- It is highly costly and time-consuming.
- It is not able for Black Box Testing.